The conflict game is the better resolver

On the same planned routes, `conflict_game.py` leaves 13.0 s of risk where `simple_resolver.py` leaves 103.5 s. It keeps working when `d_safe` rises above 3 m, where the simple rule stops removing most conflicts, and it uses less computing time. The simple rule's real advantage is mission time. Neither method guarantees eq. 20: after the repair step, conflicts remain on 8 of 30 maps with the game and 10 of 30 with the simple rule. A one-line change to the game's cost, tested here but not yet in the repository, cuts the game's count to 3 maps.

Measured over 30 maps	● Conflict game	◆ Simple rule
Risk left by the resolver alone, from 1201.0 s in the plans	**13.0 s**	103.5 s
Risk left after the Step 12 repair	**6.0 s**	20.0 s
Maps with no conflict after repair	**22 of 30**	20 of 30
Risk removed if `d_safe` is 4 m on the same routes	**97 %**	20 %
Computing time for Steps 8–12, per map	**2.9 s**	5.0 s
Mean T_{max} after repair	76.45 s	**74.63 s**
Waiting added after repair, all sinks	991.5 s	**795.0 s**

Shaded cells are the better value. Risk is the time a pair of sinks spends closer than `d_safe` = 3 m, added over all pairs. One check, `conflict_game.summary`, scores both methods.

This PDF is a static copy of the interactive report. Figure 6 shows each incident at one moment, and Figure 7 shows one map.

The problem both methods solve

Steps 1–7 plan each sink's route on its own. All sinks then leave the base at t = 0, so nothing yet keeps them apart. Across the 30 maps the plans hold 506 conflicting pairs and 1201.0 s of risk. About 80 % of that risk falls in the first 10 s, while the sinks fan out from the base.

A resolver gets the timed trajectories and may change them, as long as no sink exceeds the budget B = 80 s. Both methods read and return the same data, so `planner.py` can use either one.



Figure 1. Seed 1 before resolving: 15 sinks, 98 sensors and 16 conflicting pairs. Grey shapes are obstacles, dots are sensors and the triangle is the base. Each ring marks where a pair first comes too close, and most rings crowd around the base.

How each one decides

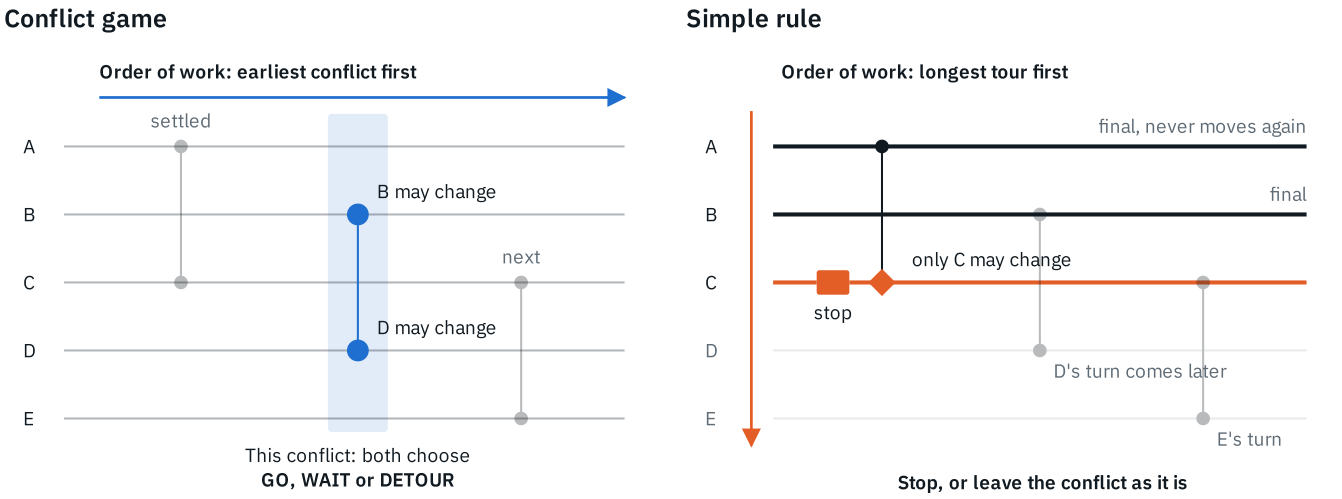


Figure 2. The same five sinks and three conflicts under each method. The game moves along time, and each sink in a conflict may yield. The simple rule moves down the list of sinks, and only the sink whose turn it is may yield, by stopping.

	Conflict game	Simple rule
Who may change	Every sink in the conflict	Only the sink later in the order
Options	GO, WAIT, DETOUR	Stop, or do nothing
Accepts a stop when	It lowers that sink's cost, even if some risk remains	The way stays clear of every earlier sink, from the stop until 5 s after the meeting
Detection	Exact closest distance per step	Five looks per step
What it guarantees	Best response stops at a pure Nash equilibrium (exact potential game)	The longest tour never changes, and paths never change
Budget B	Never exceeded	Never exceeded
Code	278 lines	75 lines

Results on 30 maps

Both methods start from identical routes on every map. Used alone, the game leaves less risk on 22 maps, the simple rule on 3, and 5 tie (Wilcoxon signed-rank $p < 0.001$). The Step 12 repair then narrows the gap, because it re-plans the sinks still in conflict. After repair the game leads on 8 maps and the simple rule on 2, with 20 ties. That lead is suggestive but not conclusive on 30 maps ($p = 0.06$).

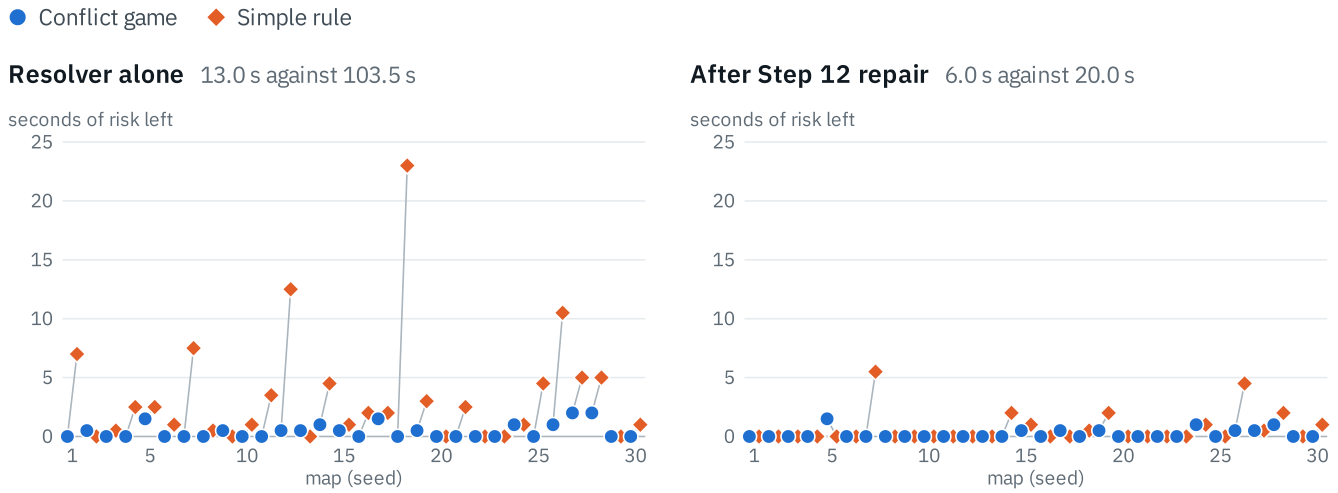


Figure 3. Seconds of risk left on each map, on one scale. Marks at zero are maps with no conflict. The table at the end of this report lists every value.

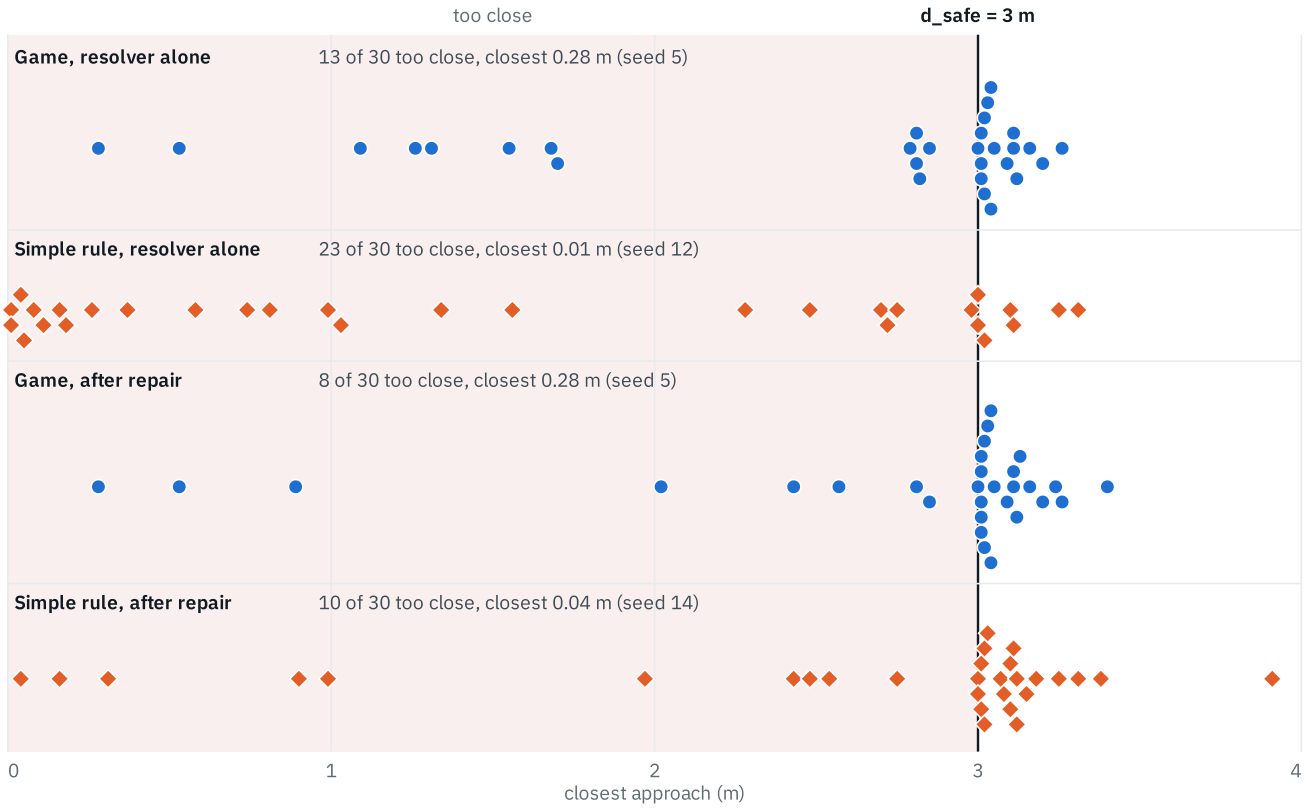


Figure 4. Closest approach on each map, one mark per map. Marks left of the line break eq. 20. Used alone, the simple rule comes within 1 m on 14 maps and the game on 2. After repair the counts are 5 and 3.

Under stress, the simple rule falls off a cliff

This test keeps each map's routes and changes only `d_safe`. The default of 3 m sits just below the point where the simple rule stops working: going to 3.25 m drops the share of risk it removes from 91 % to 33 %. By 4 m its waiting has fallen from 817.0 s to 417.5 s while the risk left has risen from 103.5 s to 1295.5 s, because it gave up 2217 times against 219 at 3 m. The game still removes 97 % at 4 m and 96 % at 5 m, then fades as traffic gets too dense.

At 2 m the two tie on risk (8.5 s), and the simple rule ends more maps with no conflict (22 against 16), though its worst miss is closer (0.02 m against 1.07 m). The cliff has a specific cause, explained in the next section, and a small fix removes it.

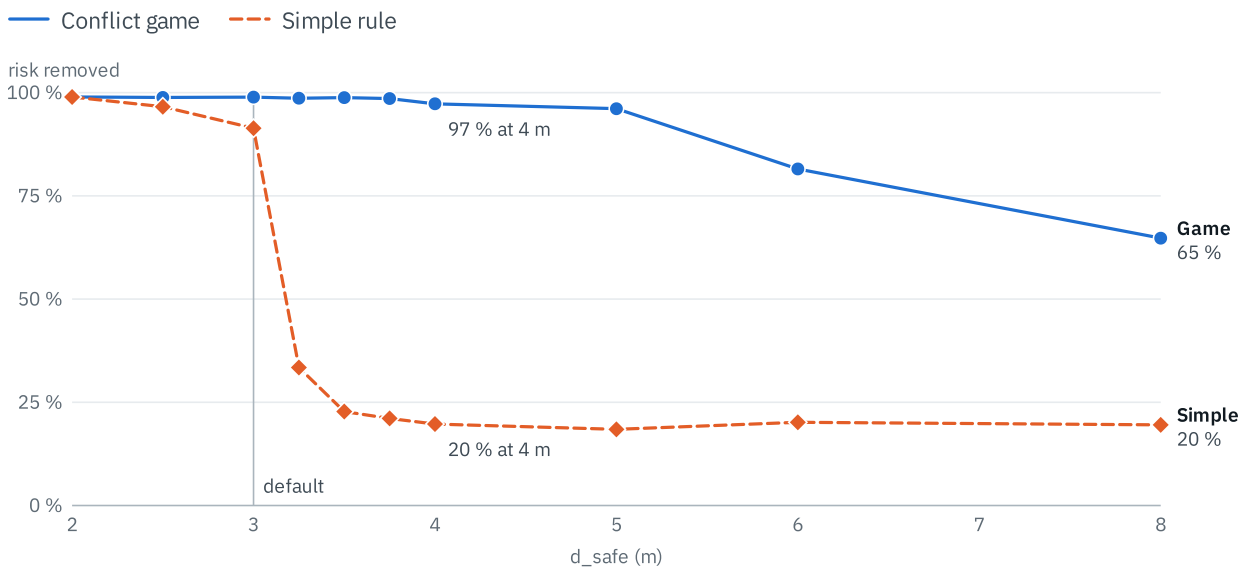


Figure 5. Share of the plan's risk removed by each resolver alone, as `d_safe` grows. Routes are not re-planned for each value, so this tests how the resolvers behave in heavier traffic and does not predict a full run at that `d_safe`.

Table for Figure 5

d_safe (m)	Risk in plans (s)	Game: risk left (s)	Game: maps clean	Game: waiting (s)	Simple: risk left (s)	Simple: maps clean	Simple: waiting (s)
2	807.0	8.5	16	486.0	8.5	22	455.0
2.5	993.5	11.5	16	703.5	34.0	12	607.5
3	1201.0	13.0	17	971.5	103.5	7	817.0
3.25	1303.0	17.5	14	1089.0	867.5	0	439.5
3.5	1412.5	17.0	16	1197.5	1091.0	0	383.5
3.75	1509.5	21.5	14	1314.0	1191.0	0	422.5
4	1614.0	43.5	8	1430.5	1295.5	0	417.5
5	2041.5	79.5	0	1955.0	1665.0	0	572.5
6	2600.0	480.5	0	2509.0	2076.0	0	945.5
8	3998.5	1409.5	0	3485.0	3217.5	0	1441.5

Why each one fails

Three real incidents, replayed from the frames each resolver produced. Discs have radius `d_safe` / 2, so two discs overlap exactly when the sinks are too close.

Seed 1: the simple rule's stop creates a chase

Shown at t = 55.0 s

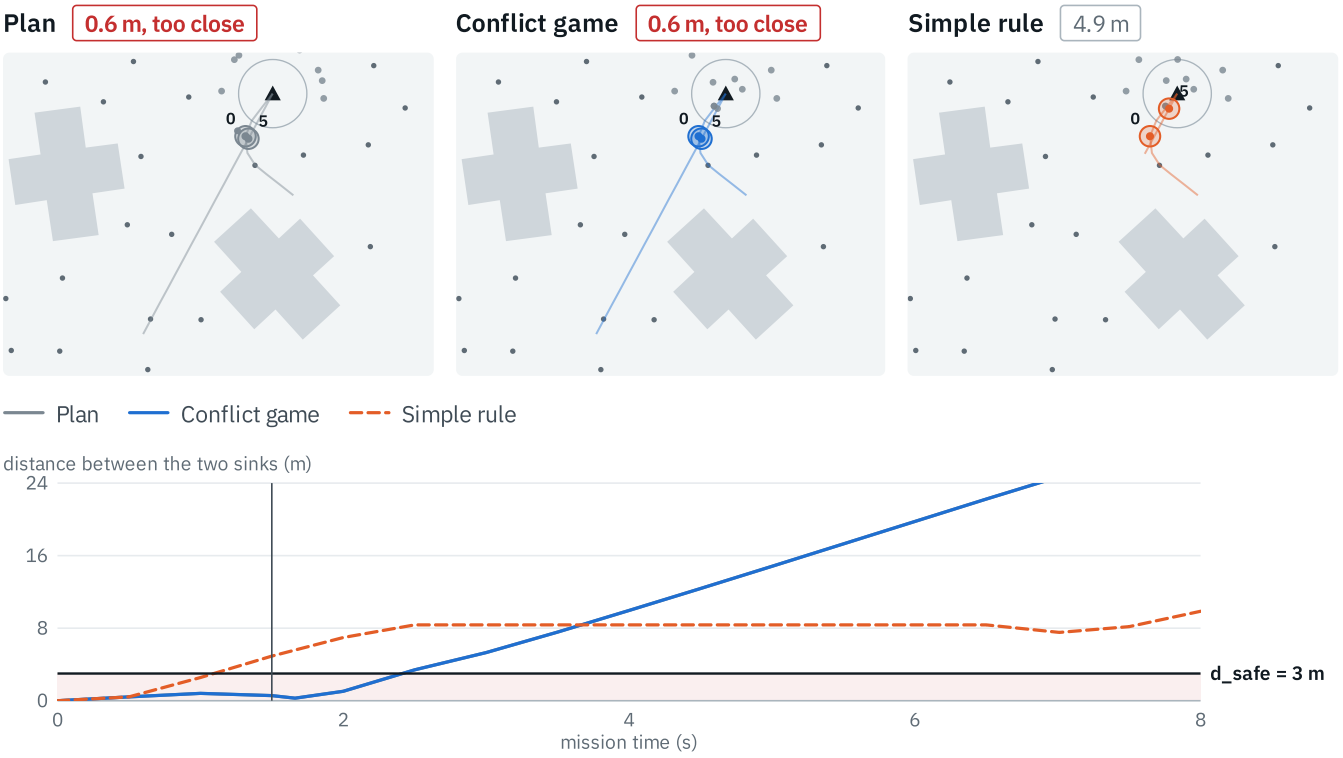


The plan keeps sinks 2 and 9 at least 9.5 m apart on the way home. The simple rule's earlier stops delay sink 2 by 2.0 s, so it leaves its last sensor just as sink 9 arrives from behind. Sink 9 had its turn first, so only sink 2 may stop, and stopping only lets sink 9 drive into it. All 37 allowed stops fail, although sink 2 has 18.9 s of budget left. The two then travel together for 7.0 s, as close as 1.56 m. Under the game, sink 2 waits only 1.0 s and the pair stays 4.5 m apart.

Figure 6a. Top: the two sinks at t = 55.0 s under each result, with other sinks as small grey dots and obstacles in grey. Bottom: distance between the two sinks over time, with the same moment marked by the vertical line.

Seed 5: the game refuses a stop

Shown at t = 1.5 s



Sinks 0 and 5 leave the base on almost the same heading and pass 0.28 m apart. Sink 0 has 0.3 s of budget left, so it can only GO. Sink 5 could WAIT, and a 6.0 s stop would clear everything in the next 6 s. The game still picks GO, because its cost adds up the whole mission:

Sink 5's options	Risk in the next 6 s	Risk over the whole mission
GO (chosen)	2.5 s	3.5 s
WAIT 6.0 s	0.0 s	4.0 s

The 4.0 s is one conflict with sink 2 at t = 52.5 s, which a later game could still solve. The simple rule clears this pair by stopping sink 5 for 8.0 s. It leaves a different conflict on this map, which the repair step then clears.

Figure 6b. Top: the two sinks at t = 1.5 s under each result, with other sinks as small grey dots and obstacles in grey. Bottom: distance between the two sinks over time, with the same moment marked by the vertical line.

Seed 24: a head-on that neither can fix

Shown at t = 31.5 s



Sinks 1 and 3 meet head-on, almost on one line. The plan has them 0.09 m apart. Each player has only GO and WAIT here, because DETOUR is not feasible at this point, and a stopped sink is simply driven through by the other. The game ends at 0.53 m and the simple rule at 0.16 m, and the repair step improves neither. The meeting lies outside the 100 × 100 m square, because the map has no outer boundary.

Figure 6c. Top: the two sinks at t = 31.5 s under each result, with other sinks as small grey dots and obstacles in grey. Bottom: distance between the two sinks over time, with the same moment marked by the vertical line.

Why the simple rule gives up

A stop counts only if the whole window is clear. The way must stay clear of every earlier sink from the stop until 5 s after the meeting. If no stop passes, the sink makes no stop at all. This happened 219 times on the 30 maps, after a median of 40 stop lengths tried, and a third sink was among the blockers in 134 of those. Budget is not the limit: the sink had a median of 25 s left, and with B = 120 s the risk left stays at 103.5 s. Accepting the stop with the lowest risk cuts it to 33.5 s.

Only one sink may yield, even when it is the wrong one. When two sinks travel the same way, the one in front cannot help by stopping, because the one behind drives into it. The turn order picks the sink to yield by tour length, whatever their positions. Same-way conflicts hold 72.0 s of the 103.5 s left, and in 18 of those 26 conflicts the sink that had to yield was in front. Reversing the order does not solve this: shortest tour first still leaves 99.0 s. Figure 6a shows this case.

Near the base, its test is stricter than the final check. Five looks flag any look outside the docking zone, while the final check asks only at the closest moment. In the plans, 520 steps are flagged only by the five looks, 509 of them in the first 5 s. The looks missed no conflict that the final check found, so the harm comes from false alarms that block good stops. With the exact test the risk left falls to 63.0 s.

The cliff in Figure 5 comes from the first and third causes together. Near the base, the earliest stop is frame 1, 2.5 m out. A sink leaving on the same heading is 5.625 m out at its first look outside the docking zone, which is 3.125 m from the stopped sink. Once `d_safe` passes about 3.125 m, stops near the base start to be refused, and 80 % of all risk sits in the first 10 s. Either fix alone removes the cliff (see the table below).

Why the game still leaves conflicts

Its cost treats later conflicts as permanent. `conflict_game.exposure` counts risk from the decision to the end of the mission, so a stop is charged for a conflict far ahead that a later game could still solve. Figure 6b shows this. Counting only the next 10 s, used alone, cuts the risk left to 6.5 s, raises maps with no conflict from 17 to 24 and lifts the worst closest approach from 0.28 m to 1.68 m, with mean T_{max} 76.00 s against 76.36 s. Through the full repair loop it gives 3.0 s left and 27 of 30 maps with no conflict, against 6.0 s and 22, with worst closest approach 2.43 m and mean T_{max} 75.65 s.

Changing the timing cannot fix a head-on meeting. When DETOUR is not feasible, the players have only GO and WAIT, and neither helps. Figure 6c shows this. Six maps keep a conflict under both methods: 15, 19, 24, 26, 27 and 28.

Most of what remains is brief and near the base. Used alone, 12 of the game's 17 remaining conflicts start in the first 10 s, as sinks cross the edge of the docking zone, and their total is only 13.0 s. DETOUR is not the source of the game's lead: it is chosen in 11 of 613 decisions, and removing it leaves 15.5 s. The lead comes from choosing who yields at each conflict and from accepting a partial improvement.

What each change does, resolver alone	Risk left, 3 m	Clean maps, 3 m	Waiting, 3 m	Risk left, 4 m	Clean maps, 4 m
Simple rule as written	103.5 s	7	817.0 s	1295.5 s	0
Simple rule, shortest tour first	99.0 s	7	887.0 s	not run	not run
Simple rule with the exact test	63.0 s	10	809.5 s	111.5 s	7
Simple rule accepting the lowest-risk stop	33.5 s	10	914.0 s	66.5 s	6
Simple rule with both fixes	31.0 s	10	840.5 s	62.0 s	6
Game without DETOUR	15.5 s	16	985.0 s	not run	not run
Game as written	13.0 s	17	971.5 s	43.5 s	8
Game with a 10 s look-ahead	**6.5 s**	**24**	985.5 s	**21.0 s**	**11**

Every variant except the two "as written" rows is a change patched in memory for this report and is not in the repository. Even with both fixes, the simple rule leaves more than twice the game's risk at 3 m.

What the game costs

The simple rule protects T_{max} by design, because the longest tour goes first and never waits. Used alone, it raised T_{max} on 2 of 30 maps, by 0.13 s on average across all 30. The game raised it on 17 maps, by 1.86 s on average. Delays under the game are also longer: 3.74 s per delayed sink against 2.74 s, with a longest delay of 19.0 s against 14.5 s. The simple rule delays more sinks, 298 of 470 against 260.

Part of the simple rule's saving comes from leaving conflicts unresolved. Per second of waiting, used alone, it removes 1.34 s of risk and the game removes 1.22 s, so the two are close in efficiency and the game simply finishes more of the job. Path length is effectively equal: the game added 10.7 m of DETOUR over all 30 maps, and total length differs by less than 1 m in 88,000 m.

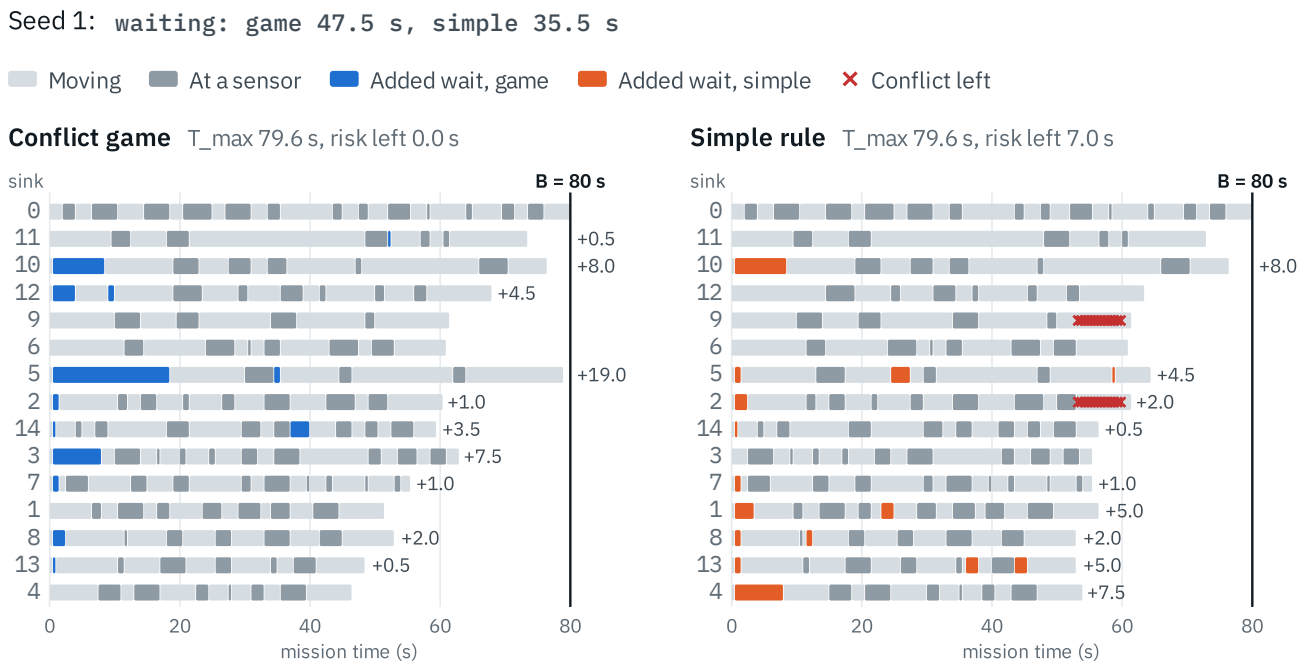


Figure 7. Each sink's mission on seed 1, resolver alone. Rows are in the simple rule's turn order, longest tour first, so its top row never waits. The number after a row is that sink's added wait in seconds. The vertical line is the budget B = 80 s.

Scorecard

The paper treats eq. 15–20 as constraints and then minimises (K, T_{max}, L_{total}) in that order, so the rows below follow the same order. A method has to satisfy the constraints before its objectives count.

Criterion	Conflict game	Simple rule	Better
Constraints			
Eq. 20: risk left, resolver alone	**13.0 s**	103.5 s	● Game
Eq. 20: risk left after repair	**6.0 s**	20.0 s	● Game
Eq. 20: maps with no conflict after repair	**22 of 30**	20 of 30	● Game
Eq. 20: worst closest approach after repair	**0.28 m**	0.04 m	● Game
Eq. 20: maps closer than 1 m after repair	**3**	5	● Game
Eq. 15–16 coverage, eq. 18 budget, $K \leq K_{max}$	Held on all 30	Held on all 30	Tie
Eq. 19: lowest obstacle clearance	1.41 m	1.41 m	Tie
Turning radius: highest curvature × R	1.014	1.014	Tie
Objectives, in the paper's order			
K, all maps	470	470	Tie
Mean T_{max} after repair	76.45 s	**74.63 s**	◆ Simple
L_{total}, all maps	88,044 m	88,045 m	Tie
Practical			
Waiting added after repair	991.5 s	**795.0 s**	◆ Simple
Risk removed at `d_safe` = 4 m	**97 %**	20 %	● Game
Computing time, Steps 8–12 per map	**2.9 s**	5.0 s	● Game
Computing time, resolver alone per map	**0.31 s**	0.49 s	● Game
Code	278 lines	**75 lines**	◆ Simple

The game's case rests on eq. 20. On the 18 maps where both methods end with no conflict, the paper's own ranking prefers the simple rule: K is equal, and its T_{max} is lower on 10 maps, higher on 1 and equal on 7 (mean 73.71 s against 75.50 s).

Recommendation

1. **Keep `conflict_game.py` as the default.** It removes more risk, degrades gradually under heavier traffic, costs less computing time, and has the equilibrium proof. Its price is about 1.8 s of T_{max}.

2. **Try the 10 s look-ahead next.** It is a one-line change to `exposure` and gave the largest gain measured here. After repair, 27 of 30 maps end with no conflict, 3.0 s of risk is left, the worst closest approach is 2.43 m and mean T_{max} falls to 75.65 s. It is better than the game as written on 7 maps and worse on 2, including seed 22, which gains a conflict. Coverage and budget held, and curvature × R reached 1.094, within the 1.15 limit. A pair's danger stays charged equally to both sinks, so the potential-game proof should carry over, but this variant has not been through `tests/run_tests.py`.
3. **If the simple rule stays as the paper's baseline, fix its two defects first.** The cliff comes from its own test near the base and from its all-or-nothing stop, and has little to do with waiting as an idea. A reviewer could object to a baseline with those defects. With both fixes it still leaves 31.0 s against the game's 13.0 s, so the conclusion holds and the comparison is fairer.
4. **Do not claim that eq. 20 holds.** Report it as measured. Even with the look-ahead, 3 of 30 maps keep a conflict, and timing changes alone cannot solve a head-on meeting when no DETOUR fits.

Running both resolvers and keeping the better result gains less than item 2: 24 of 30 maps with no conflict and 4.0 s of risk, computed from the runs above, for about 8 s of computing per map. Its mean T_{max} of 75.28 s is slightly lower.

Also found

- **Seed 22 breaks eq. 19 before either resolver runs.** Sink 13 passes 1.41 m from an obstacle against `r_safe` = 2 m. Both methods inherit it, so it does not affect the ranking. `planner.py` prints the value without flagging it and lists no problem for this map. The cause was not investigated.
- **Seeds 1–10 flatter both methods.** Each ends with no conflict on 9 of those 10 maps, as the README reports. On seeds 11–30 the game manages 13 of 20 and the simple rule 11 of 20.
- **The docking limit does not change the ranking.** Moving `base_radius` by 1e-6 m gives 12.5 s and 97.0 s left, resolver alone, against 13.0 s and 103.5 s.

How this was measured

- **Maps.** Seeds 1–30 with `default_params()`: 3008 sensors and 470 sinks in total. Each map is planned once with `rng(seed)`, as in `planner.run`, and both resolvers receive that plan.
- **Runs.** "Resolver alone" is one call of each `resolve`. "After repair" is `planner.resolve_and_check` with `rng(seed + 1000)`, the default path, which may re-plan sinks and keeps the round with the least risk.
- **Agreement with the README.** On seeds 1–10 these runs reproduce the README's comparison table: risk alone 2.5 s and 22.5 s, risk after repair 1.5 s and 5.5 s, closest 0.28 m and 0.90 m.
- **Tests.** Paired Wilcoxon signed-rank on risk per map, ignoring ties. Maps are one family from one generator, so the p-values describe these conditions only.
- **Computing time.** CPU time from `time.process_time`, with two processes running on a 2-core codespace. Compare the two methods with each other, and do not read the values as absolute speeds.
- **What-if variants.** These are patched in memory for this report. The repository code was not changed, and the variants are not implemented in it.
- **Repeat runs.** A codespace restart wiped the first run's files partway through. Everything was re-run except these totals, which are kept from the first run on the same code and seeds: `d_safe` of 2, 5, 6 and 8 m, the B sweep, the docking-limit check, shortest tour first, the game without DETOUR, and the counts of give-ups and leftover conflicts. All totals that were measured twice matched exactly.

All 30 maps

Seed	Sensors	K	Plan: pairs	Plan: risk (s)	Alone: game risk (s)	Alone: simple risk (s)	Alone: game closest (m)	Alone: simple closest (m)	Repair: game risk (s)	Repair: simple risk (s)	Repair: game closest (m)	Repair: simple closest (m)	Repair: game T_max (s)	Repair: simple T_max (s)
1	98	15	16	63.0	0.0	7.0	3.16	1.56	0.0	0.0	3.16	3.12	79.63	79.63
2	102	16	16	31.5	0.5	0.0	2.82	3.10	0.0	0.0	3.01	3.10	68.71	68.21
3	103	16	16	29.0	0.0	0.5	3.11	2.72	0.0	0.0	3.11	3.08	68.34	64.25
4	109	16	19	50.5	0.0	2.5	3.02	0.81	0.0	0.0	3.02	3.01	72.45	72.45
5	94	14	16	31.5	1.5	2.5	0.28	1.34	1.5	0.0	0.28	3.03	79.69	79.69
6	101	16	13	43.5	0.0	1.0	3.26	2.28	0.0	0.0	3.26	3.10	73.86	73.86
7	107	15	13	19.0	0.0	7.5	3.03	0.37	0.0	5.5	3.03	0.90	79.09	79.09
8	99	16	12	26.0	0.0	0.5	3.09	2.70	0.0	0.0	3.09	3.18	79.75	79.75
9	93	15	15	37.5	0.5	0.0	2.81	3.25	0.0	0.0	3.13	3.25	75.54	75.54
10	109	16	15	44.0	0.0	1.0	3.12	2.98	0.0	0.0	3.12	3.15	78.27	77.27
11	96	16	17	35.5	0.0	3.5	3.20	1.03	0.0	0.0	3.20	3.38	76.04	64.54
12	93	14	13	30.0	0.5	12.5	1.09	0.01	0.0	0.0	3.24	3.91	79.67	73.67
13	110	16	12	28.5	0.5	0.0	1.55	3.00	0.0	0.0	3.01	3.00	78.17	77.59
14	98	16	24	52.0	1.0	4.5	1.70	0.04	0.0	2.0	3.40	0.04	74.19	73.19
15	96	16	23	53.5	0.5	1.0	2.85	2.48	0.5	1.0	2.85	2.48	78.85	70.94
16	103	15	10	32.5	0.0	2.0	3.01	0.18	0.0	0.0	3.01	3.12	78.27	78.27
17	94	15	19	54.5	1.5	2.0	1.68	0.05	0.5	0.0	2.02	3.02	77.79	77.00
18	100	15	15	55.0	0.0	23.0	3.05	0.26	0.0	0.5	3.05	1.97	73.89	72.89
19	92	17	21	47.0	0.5	3.0	2.81	0.99	0.5	2.0	2.81	0.99	76.35	76.35
20	105	17	18	29.5	0.0	0.0	3.01	3.11	0.0	0.0	3.01	3.11	77.94	78.94
21	108	17	20	56.0	0.0	2.5	3.11	0.01	0.0	0.0	3.11	3.07	74.05	69.05
22	102	16	18	31.5	0.0	0.0	3.02	3.02	0.0	0.0	3.02	3.02	76.43	73.43
23	91	16	15	27.5	0.0	0.0	3.04	3.31	0.0	0.0	3.04	3.31	70.29	69.79
24	110	15	14	40.0	1.0	1.0	0.53	0.16	1.0	1.0	0.53	0.16	79.07	79.07
25	95	15	17	33.0	0.0	4.5	3.00	0.74	0.0	0.0	3.00	3.01	75.89	75.89
26	109	17	22	55.5	1.0	10.5	2.79	0.58	0.5	4.5	2.57	0.31	79.62	71.68
27	103	16	22	50.5	2.0	5.0	1.26	0.08	0.5	0.5	2.43	2.43	79.70	78.70
28	106	16	23	38.5	2.0	5.0	1.31	0.11	1.0	2.0	0.89	2.54	78.59	75.80
29	91	15	10	31.0	0.0	0.0	3.04	3.00	0.0	0.0	3.04	3.00	75.72	74.72
30	91	15	22	44.0	0.0	1.0	3.01	2.75	0.0	1.0	3.01	2.75	77.62	77.62